

UNIVERSIDADE FEDERAL DE UBERLÂNDIA – UFU
FACULDADE DE COMPUTAÇÃO – FACOM
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

CONSTRUÇÃO DE COMPILADORES
CHECKPOINT 1 — ESPECIFICAÇÃO LÉXICA E
ANALISADOR LÉXICO

Docente: Caio Henrique Corrêa Araújo
Discente: Marcelo de Almeida Maia
Data: 29 de Setembro de 2026

1. Especificação Léxica da Linguagem

Este documento apresenta a especificação léxica da linguagem desenvolvida pelo grupo para o projeto da disciplina de Construção de Compiladores. Nesta etapa, são definidas as regras para reconhecimento dos tokens da linguagem, incluindo identificadores, palavras reservadas, strings, operadores e literais numéricos.

Também são estabelecidas as regras referentes ao alfabeto de entrada, uso de maiúsculas e minúsculas, espaços em branco, comentários e desambiguação de operadores. Essas definições servirão como base para a implementação do analisador léxico.

1.1 Categorias de Token

Token	Notação (regex/EBNF)	Exemplos válidos	Regras adotadas
Identificador	letra (letra dígito " _")*	idade, x1, total_itens	Deve começar com uma letra. Após o primeiro caractere, aceita letras, dígitos e _ . A linguagem é case-sensitive. Tamanho máximo de 64 caracteres.
Palavra reservada	Mesmo padrão de identificador + consulta à tabela de palavras reservadas	if, while, int, return	Utiliza uma lista fechada de palavras reservadas. Após reconhecer o lexema, o scanner verifica se ele pertence a essa lista.

String	" (caractere válido escape)* "	"Olá", "linha 1", "João Silva"	Delimitada por aspas duplas. Serão aceitos os escapes \", \\, \n e \t. Strings não podem ocupar mais de uma linha. String não fechada gera erro léxico.
Operador	+ - * / = == != < <= > >= && !	=, ==, <=, +, &&	São permitidos operadores de um ou dois caracteres. Quando houver mais de uma possibilidade, será aplicada a regra de <i>maximal munch</i> .
Literal numérico	dígito+ ("." dígito+)?	10, 0, 3.14, 25.5	São aceitos números inteiros e decimais. Não serão utilizadas notação científica nem representação hexadecimal.

1.2 Regras Gerais da Linguagem

1.2.1 Alfabeto de entrada

O alfabeto da linguagem é composto pelos caracteres que podem aparecer na formação dos tokens reconhecidos pelo analisador léxico.

São considerados válidos:

- Letras maiúsculas e minúsculas: A-Z e a-z;
- Dígitos: 0-9;
- Símbolo _ para composição de identificadores;
- Aspas duplas " para delimitação de strings;
- Caracteres utilizados nos operadores: +, -, *, /, =, !, <, >, & e |;

- Delimitadores e símbolos auxiliares: (,), {, }, , e ;;
- Caracteres de espaço em branco: espaço, tabulação e quebra de linha.

Dentro de strings também são permitidos caracteres comuns e as sequências de escape definidas nesta especificação.

1.2.2 Sensibilidade a maiúsculas e minúsculas

A linguagem é case-sensitive, portanto letras maiúsculas e minúsculas são consideradas diferentes. Dessa forma: idade, Idade e IDADE representam três identificadores distintos.

A mesma regra é aplicada às palavras reservadas. Por exemplo, while é uma palavra reservada, enquanto While e WHILE podem ser reconhecidos como identificadores, desde que atendam às demais regras definidas.

1.2.3 Espaços em branco e comentários

Espaços em branco, tabulações e quebras de linha são utilizados para separar tokens quando necessário e não são retornados como tokens pelo analisador léxico.

A linguagem aceita dois tipos de comentário:

// Comentário de uma linha

O comentário iniciado por // termina ao final da linha.

Também são aceitos comentários de bloco:

`/*`

Comentário

de bloco

`*/`

O comentário de bloco começa com `/*` e termina com `*/`. Comentários de bloco aninhados não são permitidos.

Caso o arquivo termine antes do fechamento de um comentário de bloco, o scanner deverá reportar um erro léxico.

1.2.4 Regra de desambiguação de operadores

Para operadores que possuem formas simples e compostas, será utilizada a regra de maximal munch. O analisador léxico sempre tentará reconhecer o maior prefixo válido antes de determinar o token.

Por exemplo, ao encontrar: **idade >= 18**

o scanner deve reconhecer `>=` como um único operador, em vez de reconhecer separadamente `>` e `=`.

Da mesma forma: `==` `!=` `<=` `>=` `&&` `||`

devem ser reconhecidos como operadores compostos sempre que a sequência completa estiver presente.

1.2.5 Palavras reservadas

A linguagem possui a seguinte lista fechada de palavras reservadas:

Palavra	Finalidade
int	Tipo inteiro
double	Tipo numérico de ponto flutuante
bool	Tipo booleano
char	Tipo caractere
string	Tipo cadeia de caracteres
void	Indica ausência de valor de retorno
if	Estrutura de seleção
else	Alternativa da estrutura de seleção
while	Estrutura de repetição
return	Retorno de função

Palavra**Finalidade**

true

Valor booleano verdadeiro

false

Valor booleano falso